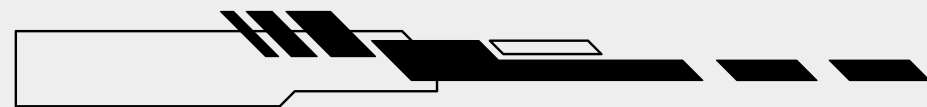


COUNTING SORT

ANALISE EXPERIMENTAL



DISCENTES:
VINICIUS MARTINS
YAN TEIXEIRA





ALGORITMO - COUNTING SORT

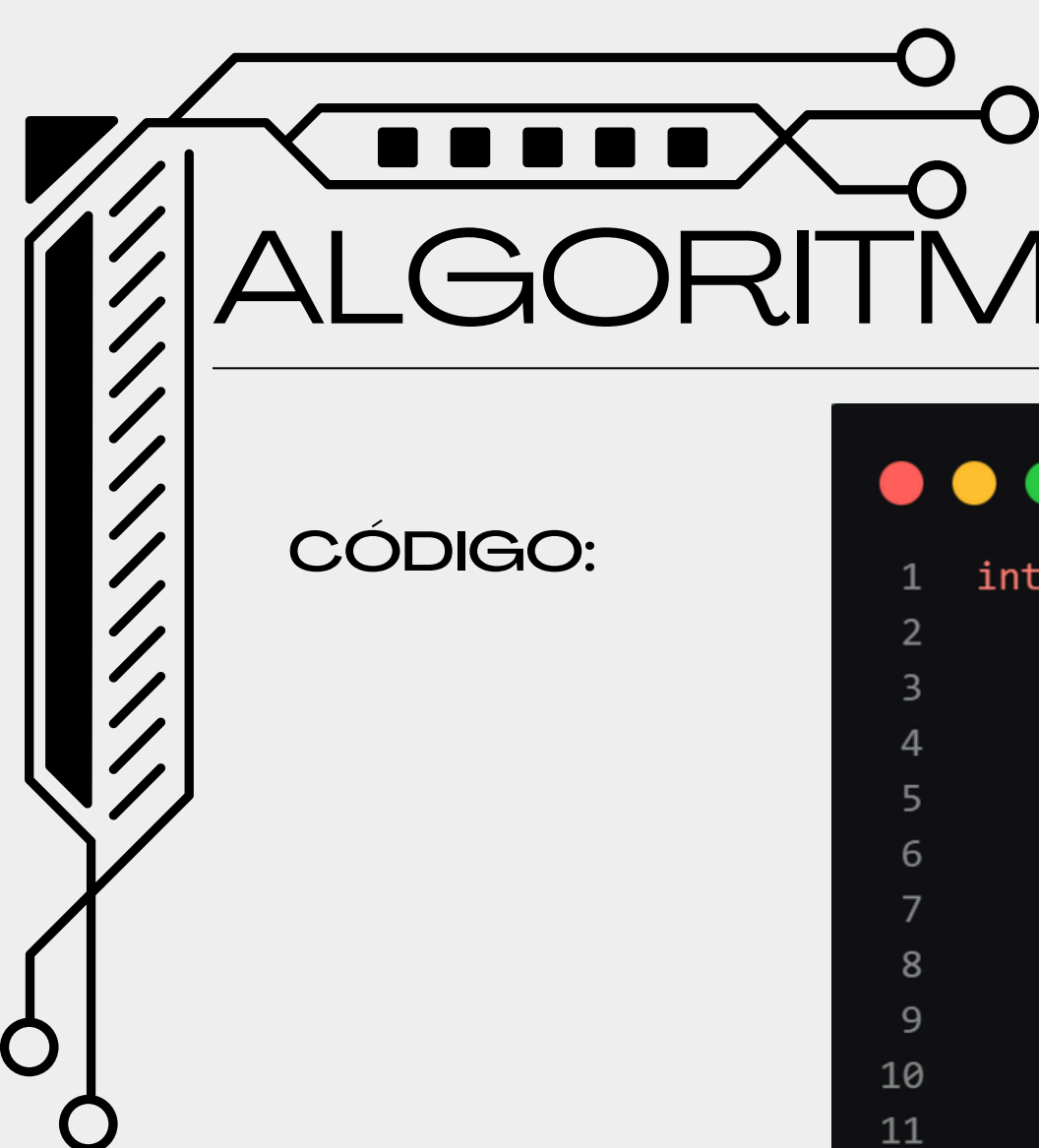
DEFINIÇÃO

O Counting Sort é um algoritmo de ordenação linear que, ao contrário de métodos como QuickSort ou MergeSort, não utiliza comparações entre os elementos. Em vez disso, ele funciona contando o número de ocorrências de cada valor único no conjunto de dados.

FUNCIONAMENTO

- **Contar:** O algoritmo percorre os dados originais e anota (em um array auxiliar) quantas vezes cada número específico aparece.
- **Soma Cumulativa:** Ele soma essas contagens sequencialmente. O resultado dessa matemática simples indica a posição exata (o índice) onde cada número deverá ser colocado no array final.
- **Distribuir:** Ele pega os números do array original e os coloca diretamente nas posições calculadas em um novo array ordenado. A cada número posicionado, ele atualiza a vaga para o próximo número igual (caso exista).

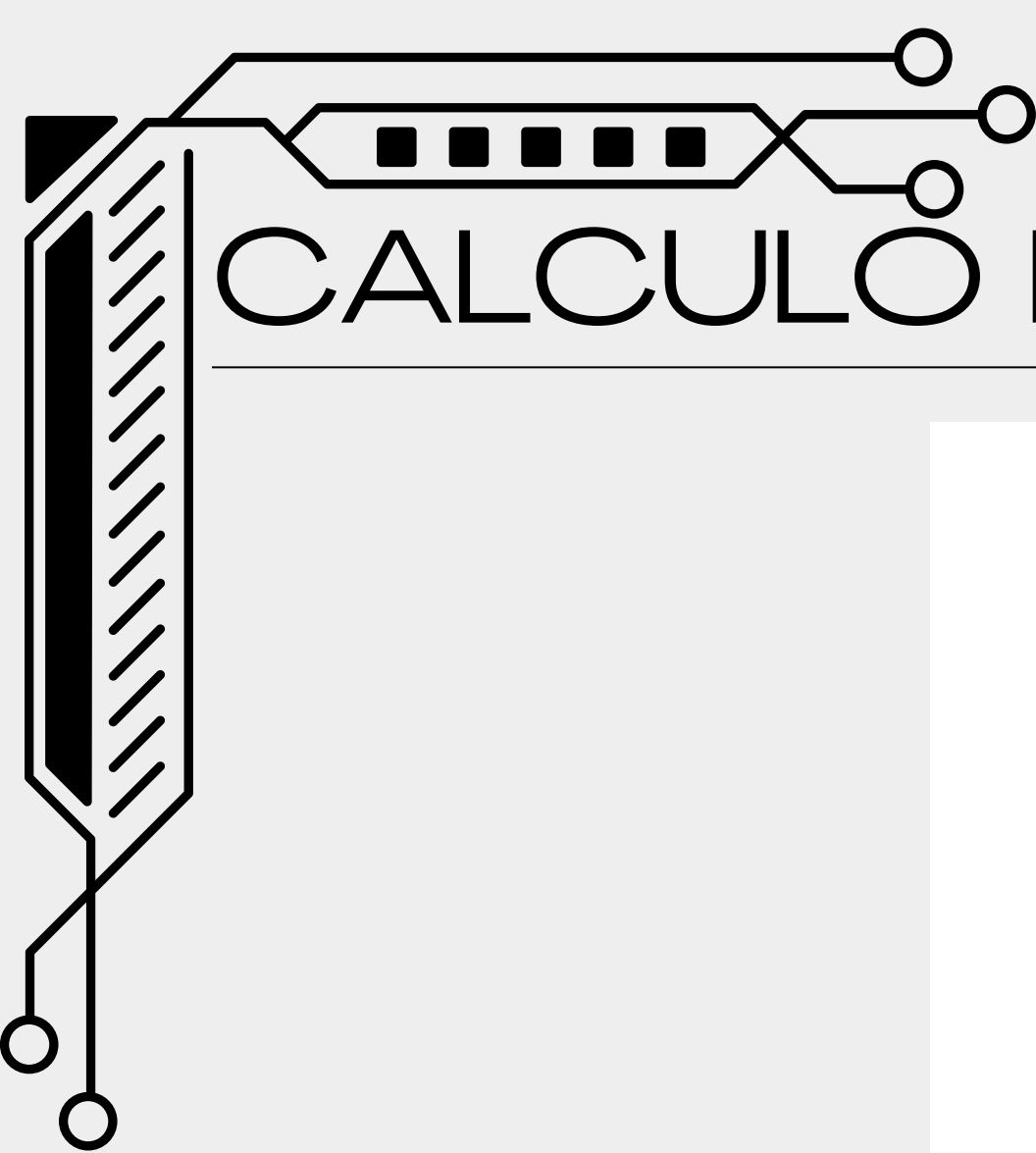
Link Visualgo: <https://visualgo.net/en/sorting>



ALGORITMO - COUNTING SORT

CÓDIGO:

```
1  int counting_sort(int *vetor, size_t n, int valor_maximo) {
2      if (n == 0) return 0;
3      if (valor_maximo < 0) return -1;
4
5      size_t k = (size_t)valor_maximo + 1;
6
7      int *contagem = (int *)malloc(k * sizeof(int));
8      if (!contagem) return -1;
9      memset(contagem, 0, k * sizeof(int));
10
11     for (size_t i = 0; i < n; i++) {
12         int v = vetor[i];
13         contagem[v]++;
14     }
15
16     size_t pos = 0;
17     for (int v = 0; v <= valor_maximo; v++) {
18         int c = contagem[v];
19         for (int j = 0; j < c; j++) {
20             vetor[pos++] = v;
21         }
22     }
23
24     free(contagem);
25     return 0;
26 }
```



CALCULO DO AGORITMO

1 Análise da Função de Custo

Para um array de entrada A com n elementos, onde cada elemento está no intervalo $[0, k]$, o custo total $T(n, k)$ é determinado pela soma das operações fundamentais do algoritmo:

1. **Inicialização:** Zerar o array de contagem de tamanho $k + 1$.

$$\text{Custo} = \Theta(k) \quad (1)$$

2. **Contagem de Frequência:** Percorrer o array de entrada uma vez para registrar as ocorrências.

$$\text{Custo} = \Theta(n) \quad (2)$$

3. **Soma Cumulativa:** Calcular as posições prefixadas no array de contagem para garantir a estabilidade.

$$\text{Custo} = \Theta(k) \quad (3)$$

4. **Construção do Array de Saída:** Mover os elementos para suas posições ordenadas no array final.

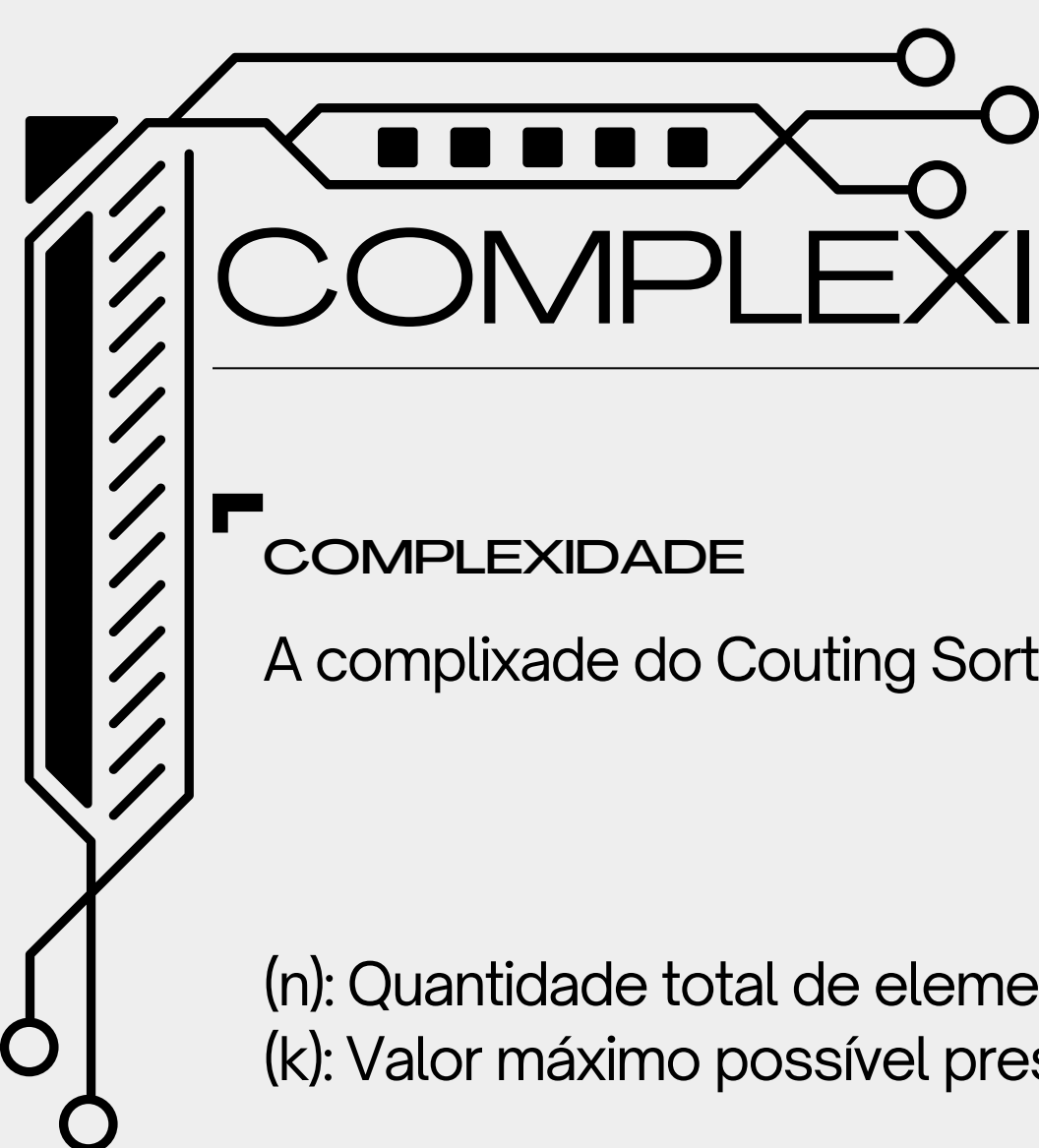
$$\text{Custo} = \Theta(n) \quad (4)$$

1.1 Equação Simplificada

A função de custo total é a soma linear desses componentes:

$$T(n, k) = c_1 \cdot k + c_2 \cdot n + c_3 \cdot k + c_4 \cdot n \quad (5)$$

$$T(n, k) = \Theta(n + k) \quad (6)$$



COMPLEXIDADE - COUNTING SORT

COMPLEXIDADE

A complexidade do Counting Sort é:

$$O(n + k)$$

(n): Quantidade total de elementos no array (tamanho da entrada).;

(k): Valor máximo possível presente nos dados (intervalo).

PIOR CASO

Ocorre quando o valor máximo (k) é desproporcionalmente gigantesco em relação à quantidade de elementos (n). Ou seja, $k \gg n$.

k dominaria n, tornando o algoritmo ineficiente comparado a outros

MELHOR CASO

Ocorre quando a amplitude dos dados (k) é muito menor ou proporcional à quantidade de elementos (n). Ou seja, k menor ou igual a n.

$$O(n)$$



METODOLOGIA

13 EXECUÇÕES POR TAMANHO

Para cada tamanho de entrada testado no experimento, o algoritmo de ordenação é executado repetidas vezes.

Isso cria uma amostra estatística sólida

RELÓGIO: `CLOCK_GETTIME()` (NANOSEGUNDOS)

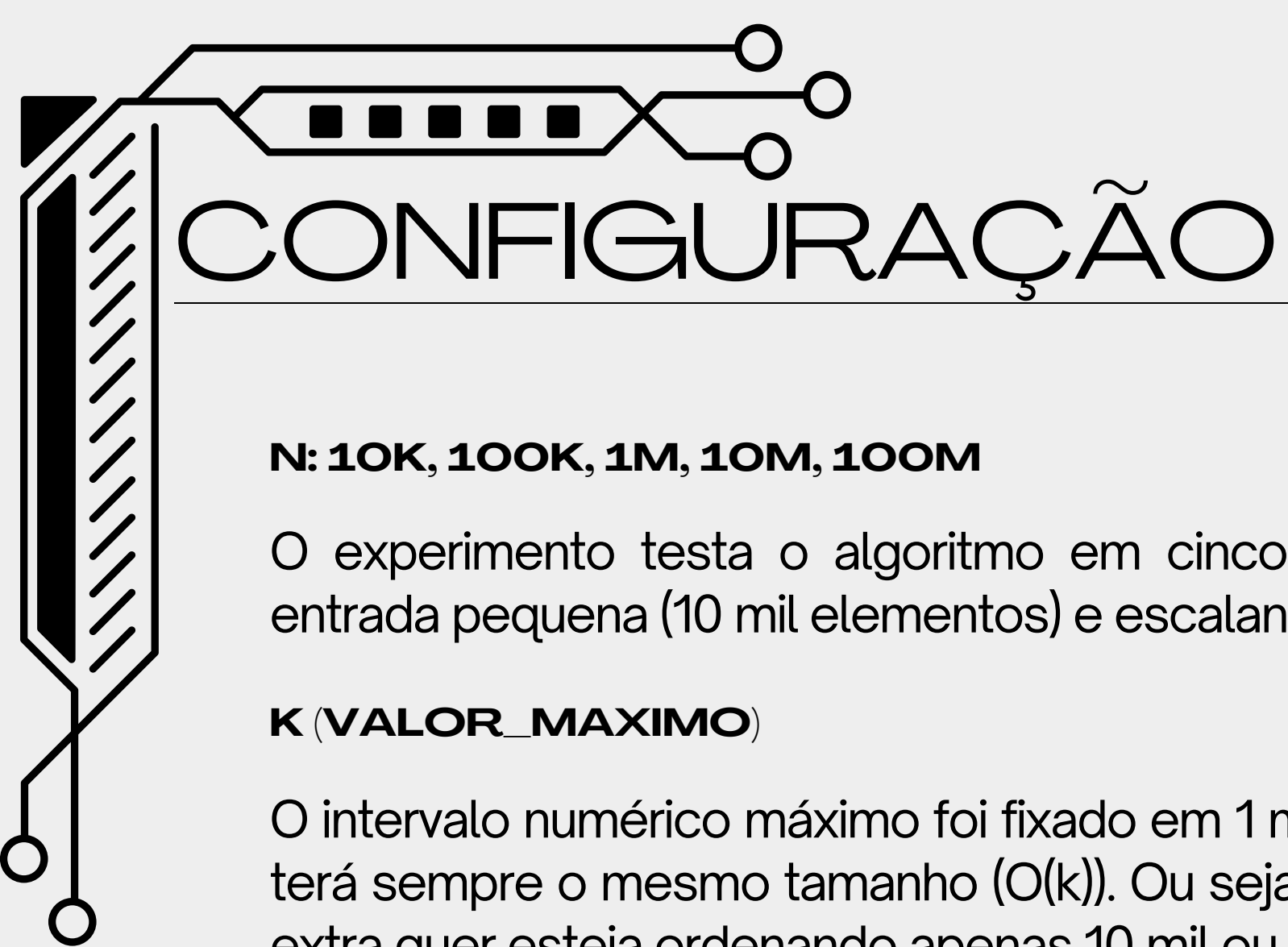
A aferição do tempo de CPU é feita utilizando a função da API POSIX `clock_gettime(CLOCK_MONOTONIC)`.

DESCARTA MIN/MAX, MÉDIA DAS 11

Aplica a média aparada (descartando o maior e o menor tempo das 13 execuções) para ignorar variações do sistema operacional e obter uma métrica mais estável baseada em 11 rodadas

SEED FIXA (REPRODUTIBILIDADE)

A geração dos conjuntos de números aleatórios para os testes utiliza a instrução `srand(42)`.



CONFIGURAÇÃO

N: 10K, 100K, 1M, 10M, 100M

O experimento testa o algoritmo em cinco cenários muito diferentes, começando com uma entrada pequena (10 mil elementos) e escalando até conjuntos massivos (100 milhões).

K (VALOR_MAXIMO)

O intervalo numérico máximo foi fixado em 1 milhão. Isso garante que o array auxiliar de contagem terá sempre o mesmo tamanho ($O(k)$). Ou seja, o algoritmo gastará o mesmo esforço de memória extra quer esteja ordenando apenas 10 mil ou 100 milhões de itens.

POR QUÊ? DEMONSTRAR $O(N + K)$

Como a complexidade é $O(n + k)$ e o k está congelado em 1 milhão, o experimento consegue provar na prática dois cenários cruciais:

Quando n é pequeno (ex: 10K): O valor de k (1M) é muito maior que n . Aqui, k domina o custo.

Quando n é massivo (ex: 100M): O valor de n ultrapassa largamente o k . Aqui, a influência do k some, e o comportamento do algoritmo escala de forma linear, mostrando sua verdadeira força e se aproximando da eficiência ideal de $O(n)$.



RESULTADO - TABELA

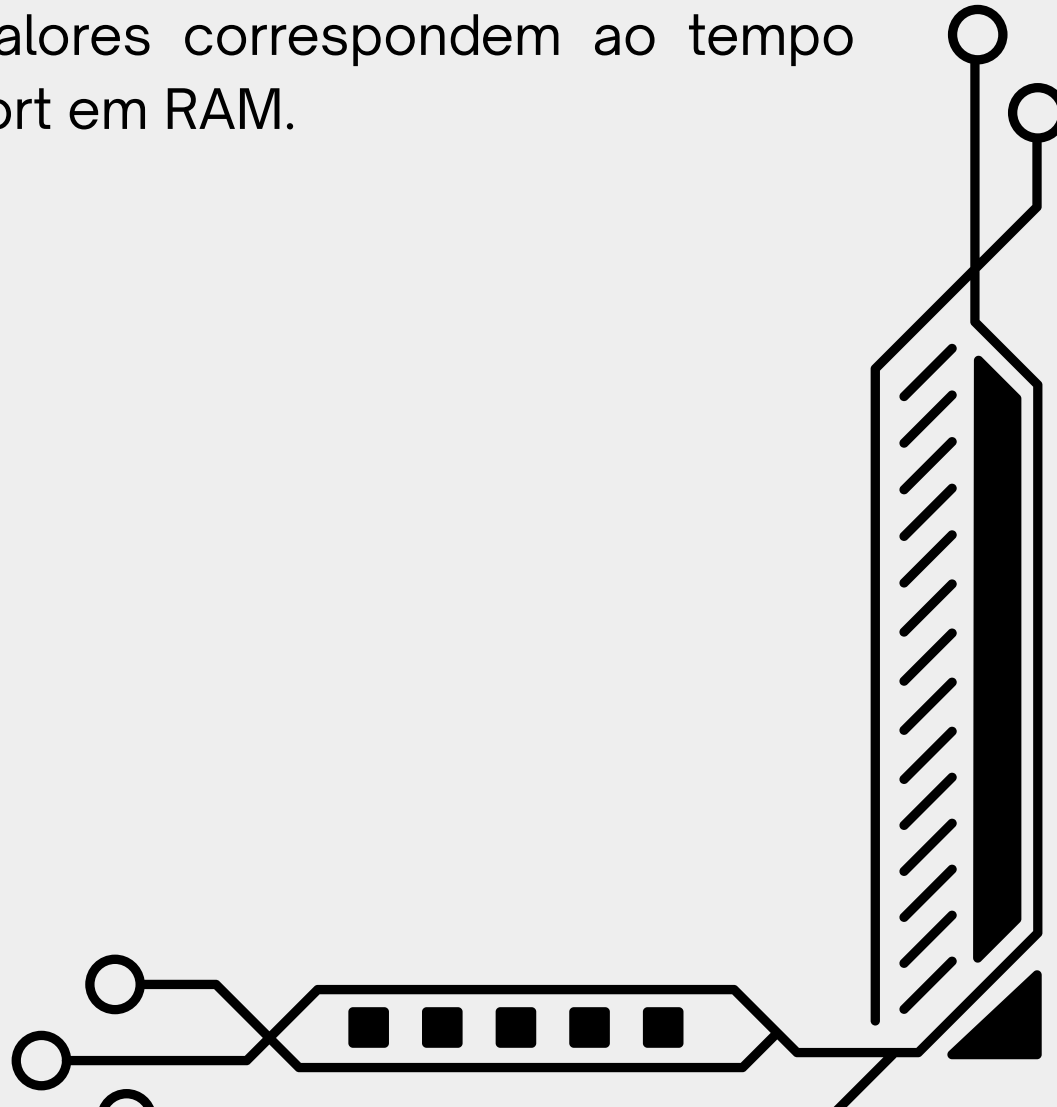
ESPECIFICAÇÕES DA MÁQUINA DE TESTE:

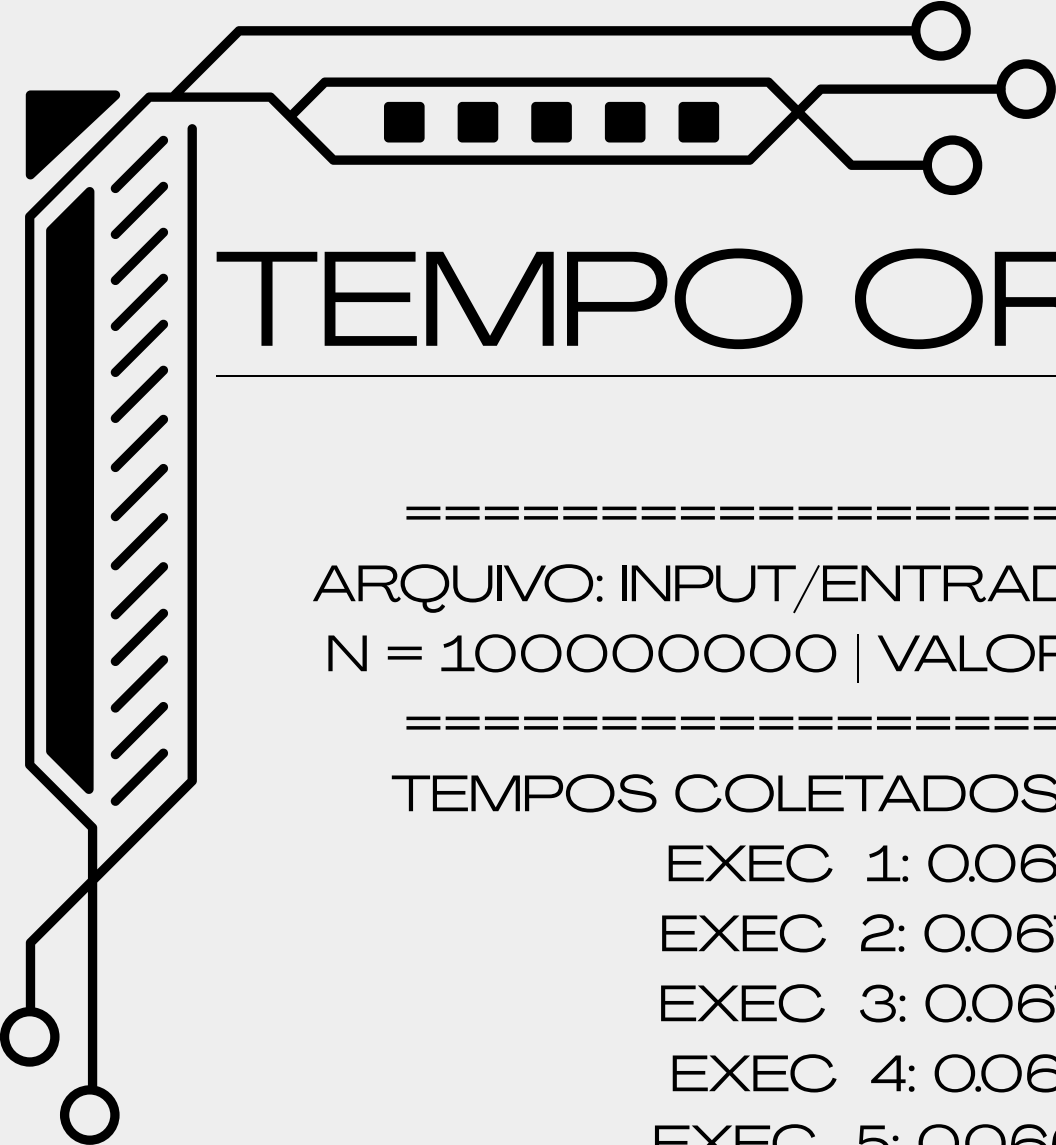
CPU: **AMD RYZEN 7 5800H**
RAM: **16G**
ARMAZENAMENTO: **SDD**
SO: **LINUX (OU AMBIENTE LINUX VIA WSL)**

n	Tempo médio (s)	Min (s)	Max (s)
10.000	0.000813754	0.000579235	0.002003109
100.000	0.001229113	0.001209774	0.001504882
1.000.000	0.006219676	0.006068867	0.006990126
10.000.000	0.018250022	0.018037222	0.018962097
100.000.000	0.118435262	0.117355991	0.120411385

Obs: esse valores correspondem ao tempo do couting sort em RAM.

Resultados da tabela





TEMPO ORDENAÇÃO VS K

=====

ARQUIVO: INPUT/ENTRADA_1000000000.TXT
N = 1000000000 | VALOR_MAXIMO = 10000

=====

TEMPOS COLETADOS (13 EXECUÇÕES):

EXEC 1: 0.068275318 S
EXEC 2: 0.067037190 S
EXEC 3: 0.067296492 S
EXEC 4: 0.067196175 S
EXEC 5: 0.066500057 S
EXEC 6: 0.066854688 S
EXEC 7: 0.066858074 S
EXEC 8: 0.066651365 S
EXEC 9: 0.067421819 S
EXEC 10: 0.067210942 S
EXEC 11: 0.067071737 S
EXEC 12: 0.067198006 S
EXEC 13: 0.066839558 S

DESCARTADOS: MIN=0.066500 S | MAX=0.068275 S
TEMPO MÉDIO (11 VALORES): 0.067057 S

=====

ARQUIVO: INPUT/ENTRADA_1000000000.TXT
N = 1000000000 | VALOR_MAXIMO = 10000000

=====

TEMPOS COLETADOS (13 EXECUÇÕES):

EXEC 1: 0.116096929 S
EXEC 2: 0.120195304 S
EXEC 3: 0.119446991 S
EXEC 4: 0.118500887 S
EXEC 5: 0.119036310 S
EXEC 6: 0.118149738 S
EXEC 7: 0.118463807 S
EXEC 8: 0.126377858 S
EXEC 9: 0.119674467 S
EXEC 10: 0.118312764 S
EXEC 11: 0.118356375 S
EXEC 12: 0.118597157 S
EXEC 13: 0.118414555 S

DESCARTADOS: MIN=0.116096 S | MAX=0.126377 S
TEMPO MÉDIO (11 VALORES): 0.118831669 S

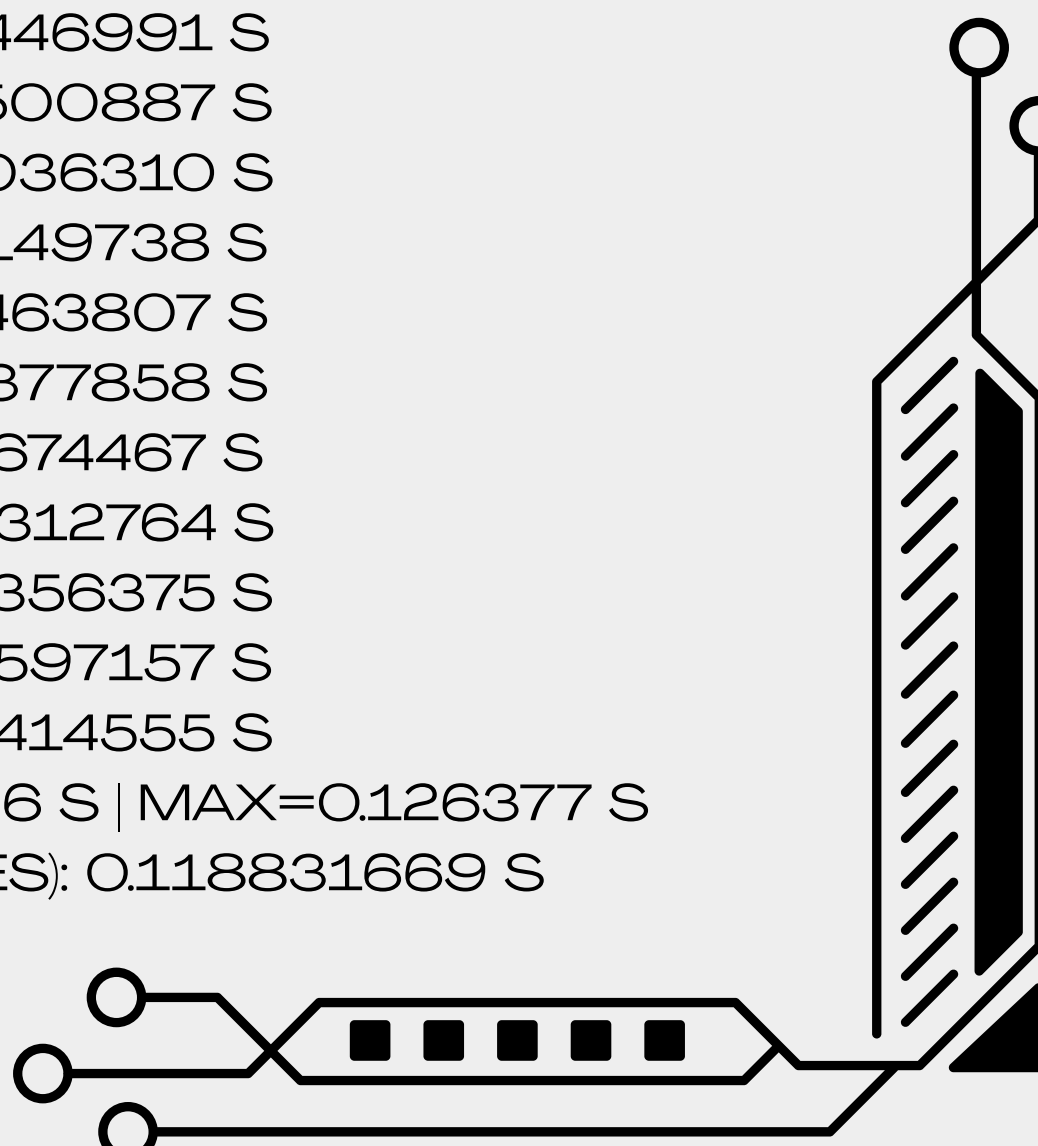
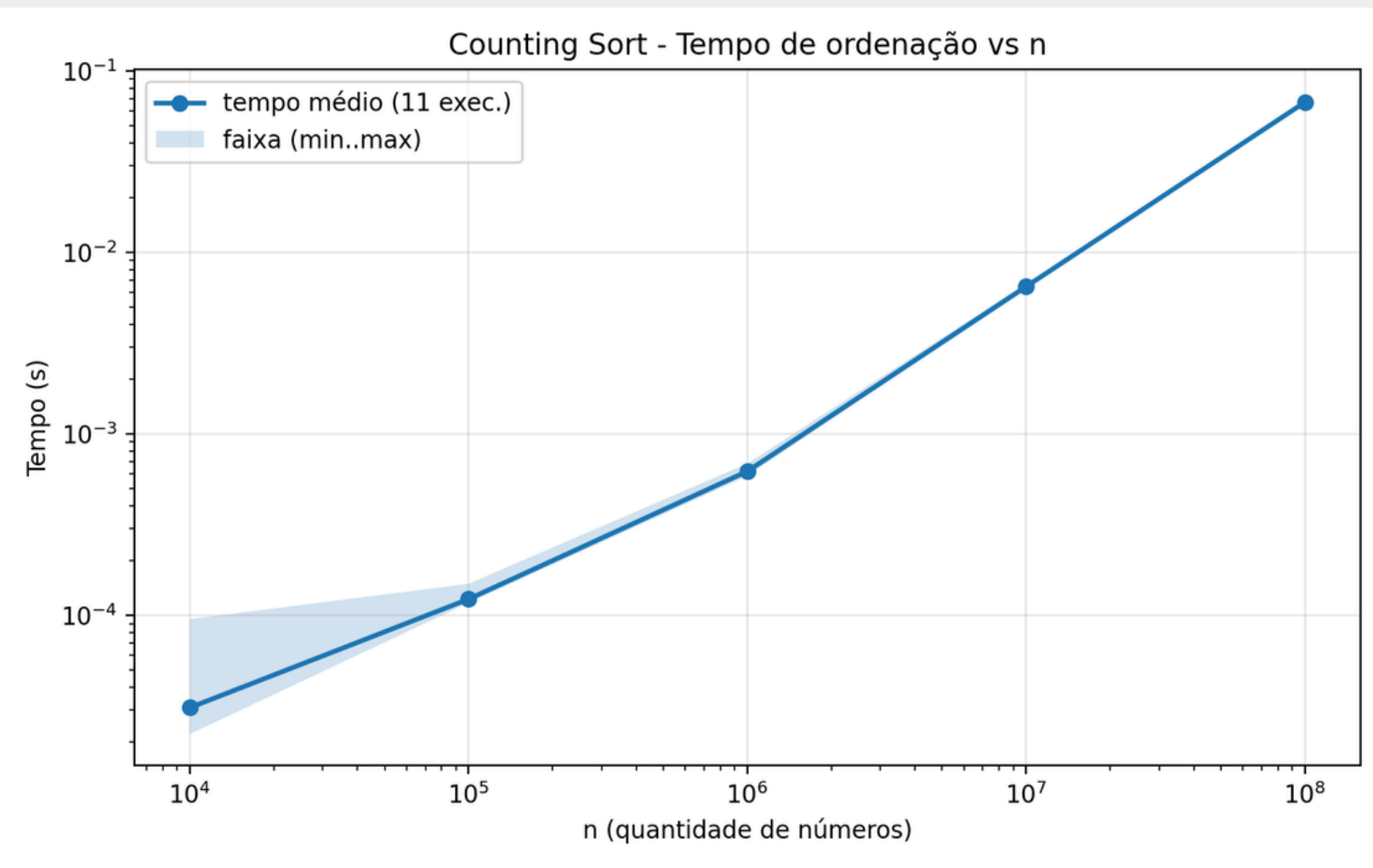
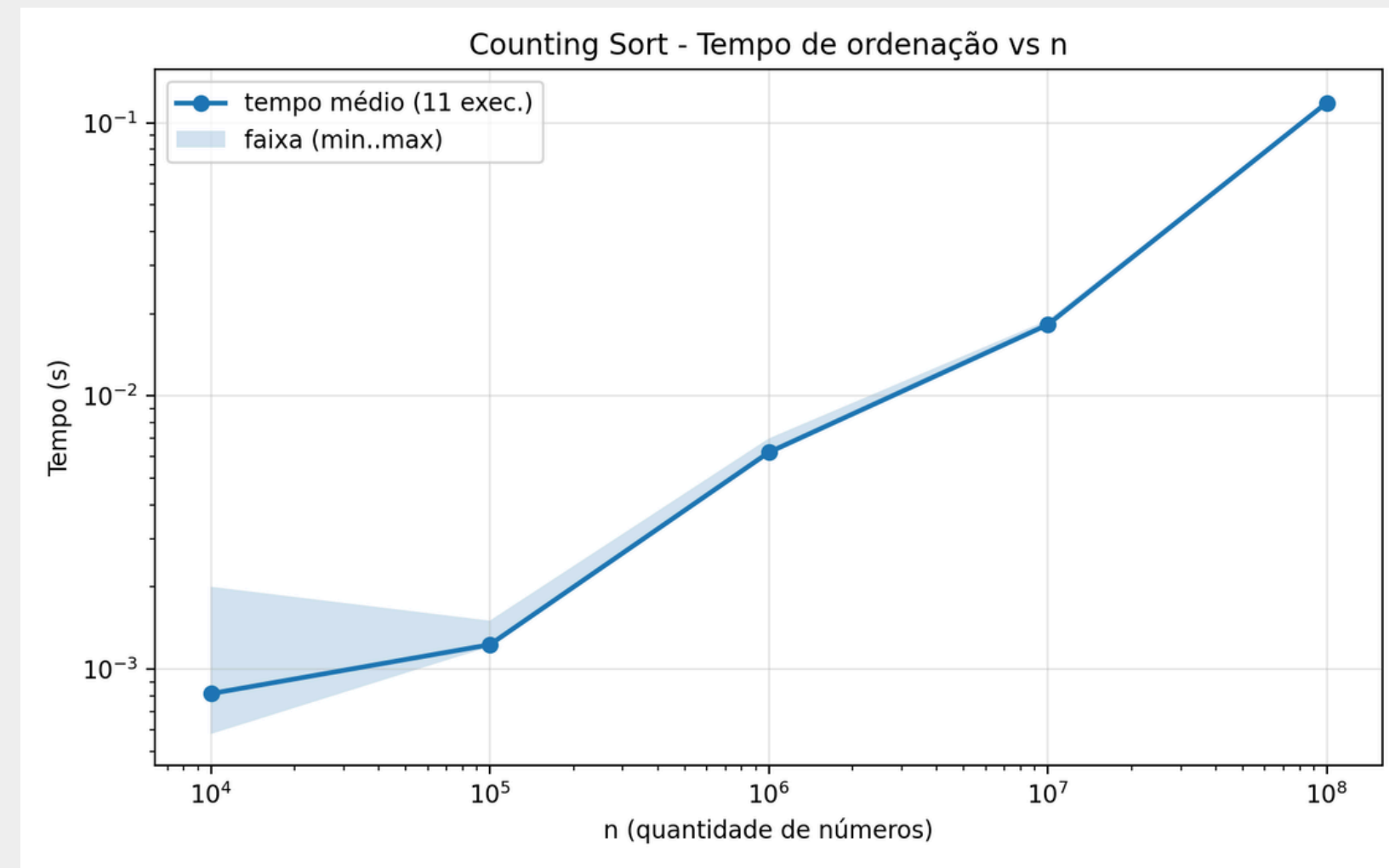


GRÁFICO TEMPO VS N



K: 10000



K: 10000000



CONSUMO MEMÓRIA VS K

COM K = 10.000:

O PICO DE MEMÓRIA REGISTRADO FOI DE ~5,5 MB (10K), ~5,5 MB (100K),
~9,9 MB (1M), ~83,6 MB (10M) E ~786,8 MB (100M).

COM K = 1.000.000:

O PICO DE MEMÓRIA REGISTRADO FOI DE ~5,6 MB (10K), ~6,2 MB (100K),
~13,2 MB (1M), ~83,6 MB (10M) E ~786,8 MB (100M).

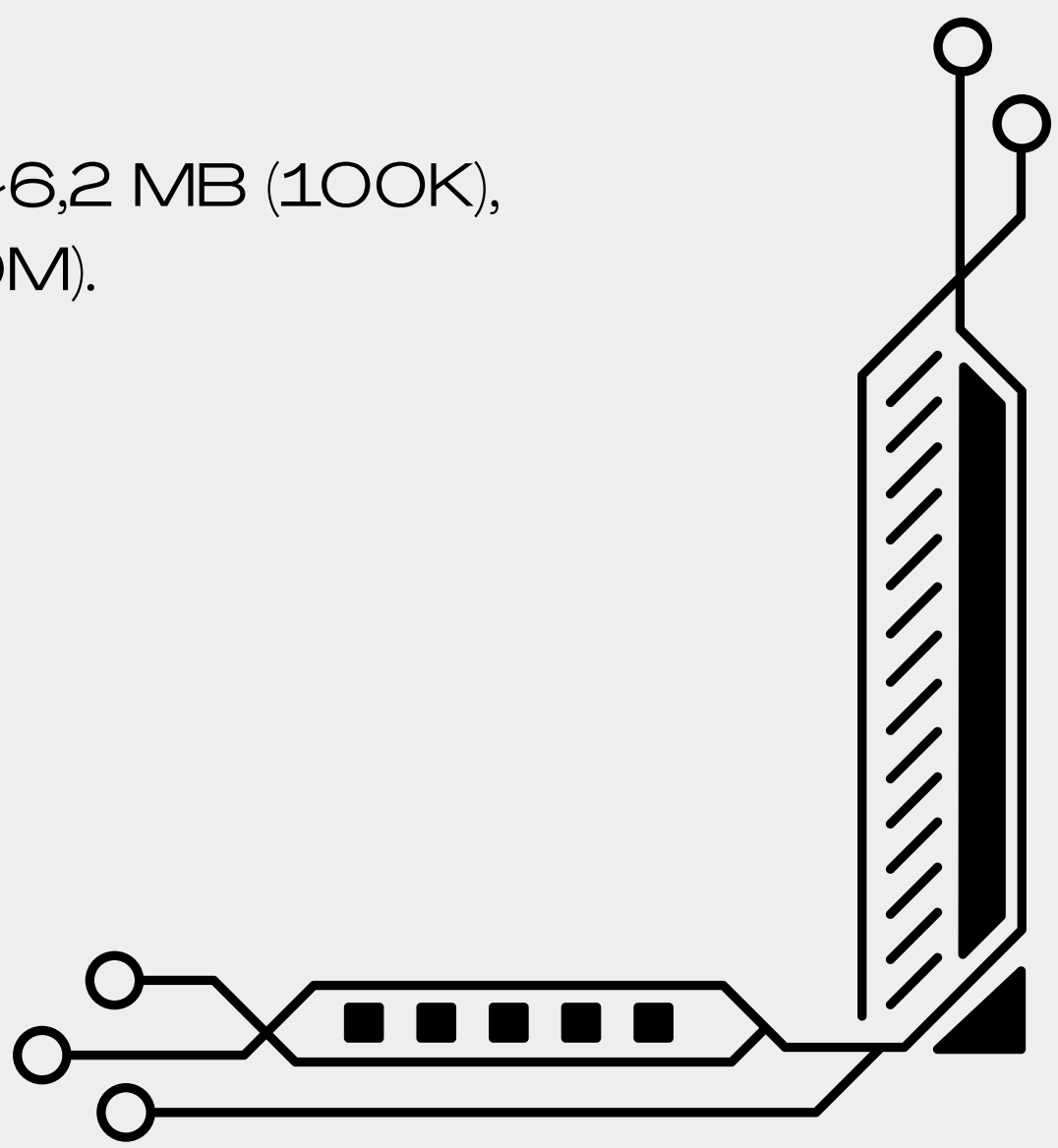
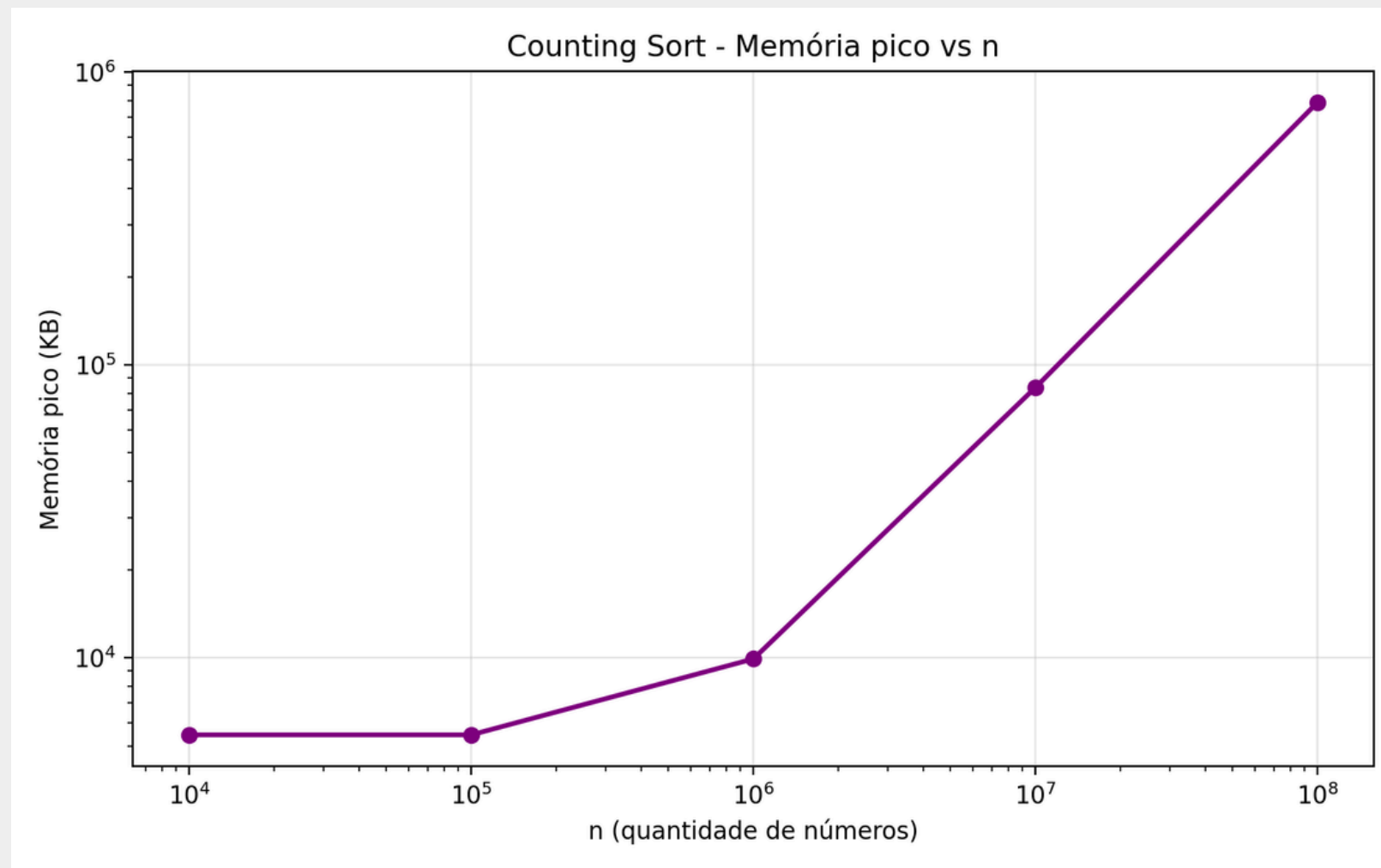
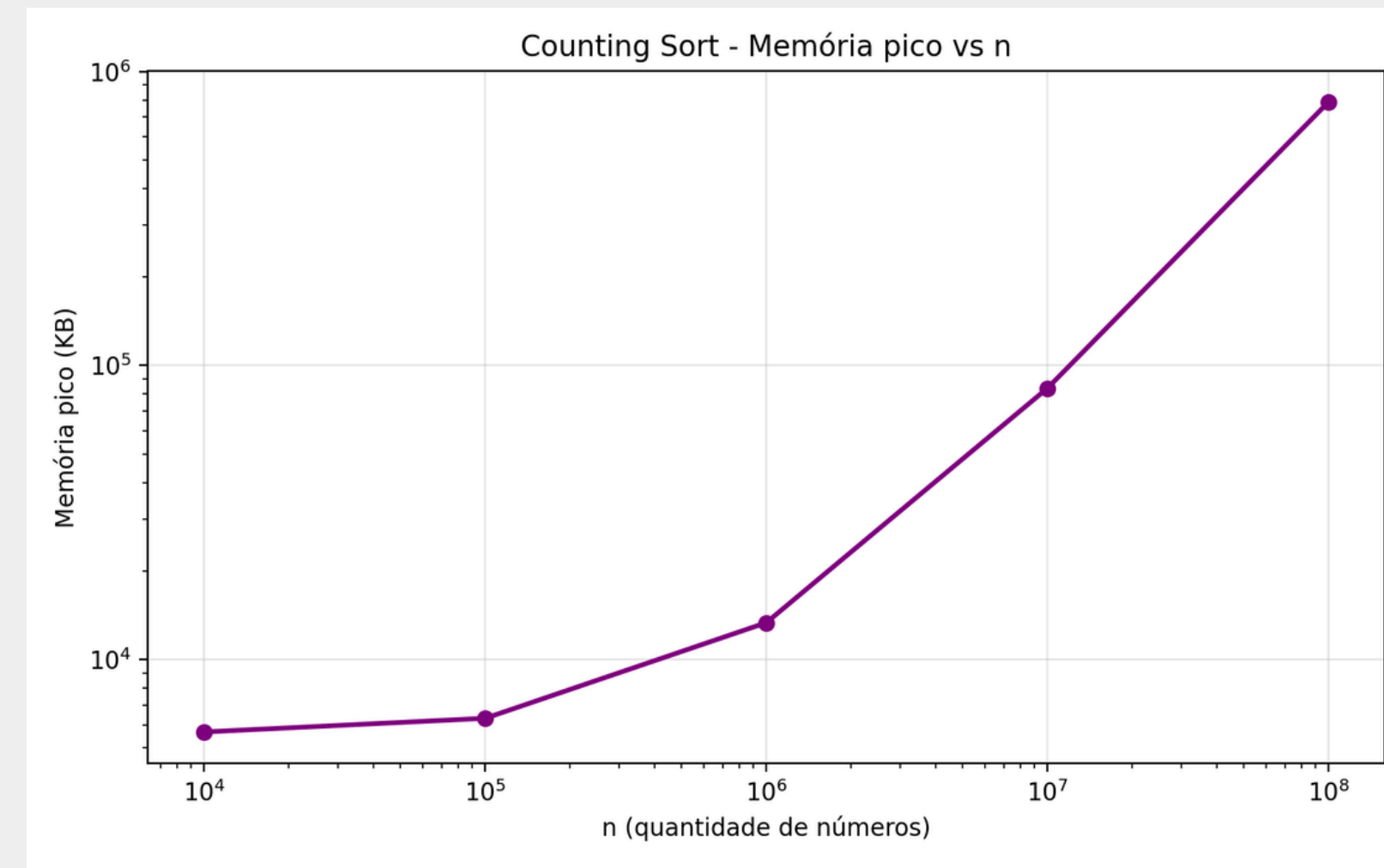


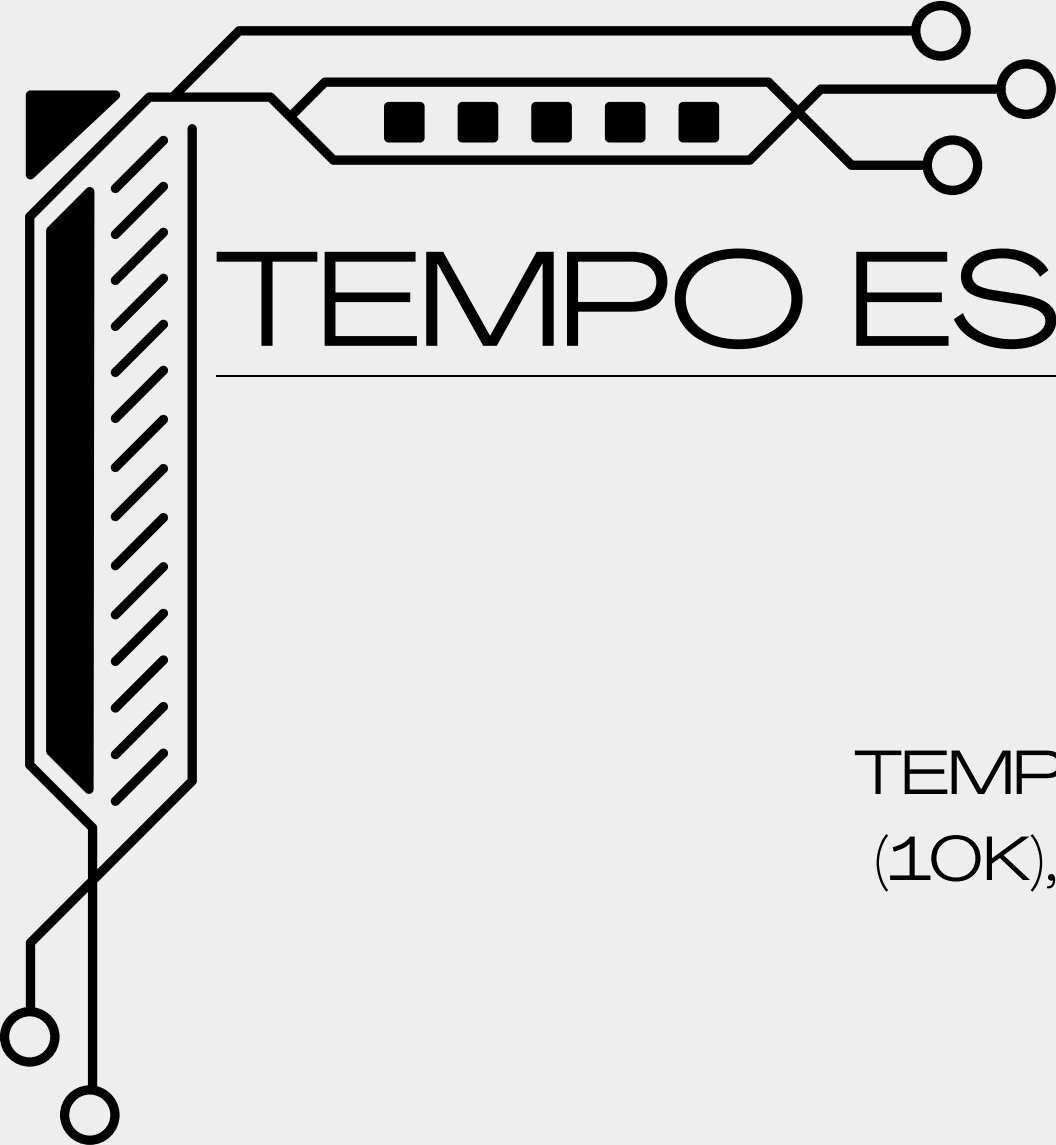
GRÁFICO MEMÓRIA VS N



K: 10000



K: 10000000



TEMPO ESCRITA VS K

K = 10.000:

TEMPO DE ESCRITA DA SAÍDA REGISTRADO: ~0,000598 S
(10K), ~0,003840 S (100K), ~0,039338 S (1M), ~0,428987 S
(10M) E ~3,925957 S (100M).

K = 1.000.000:

TEMPO DE ESCRITA DA SAÍDA REGISTRADO: ~0,001038 S
(10K), ~0,008028 S (100K), ~0,048467 S (1M), ~0,440948 S
(10M) E ~4,440639 S (100M).

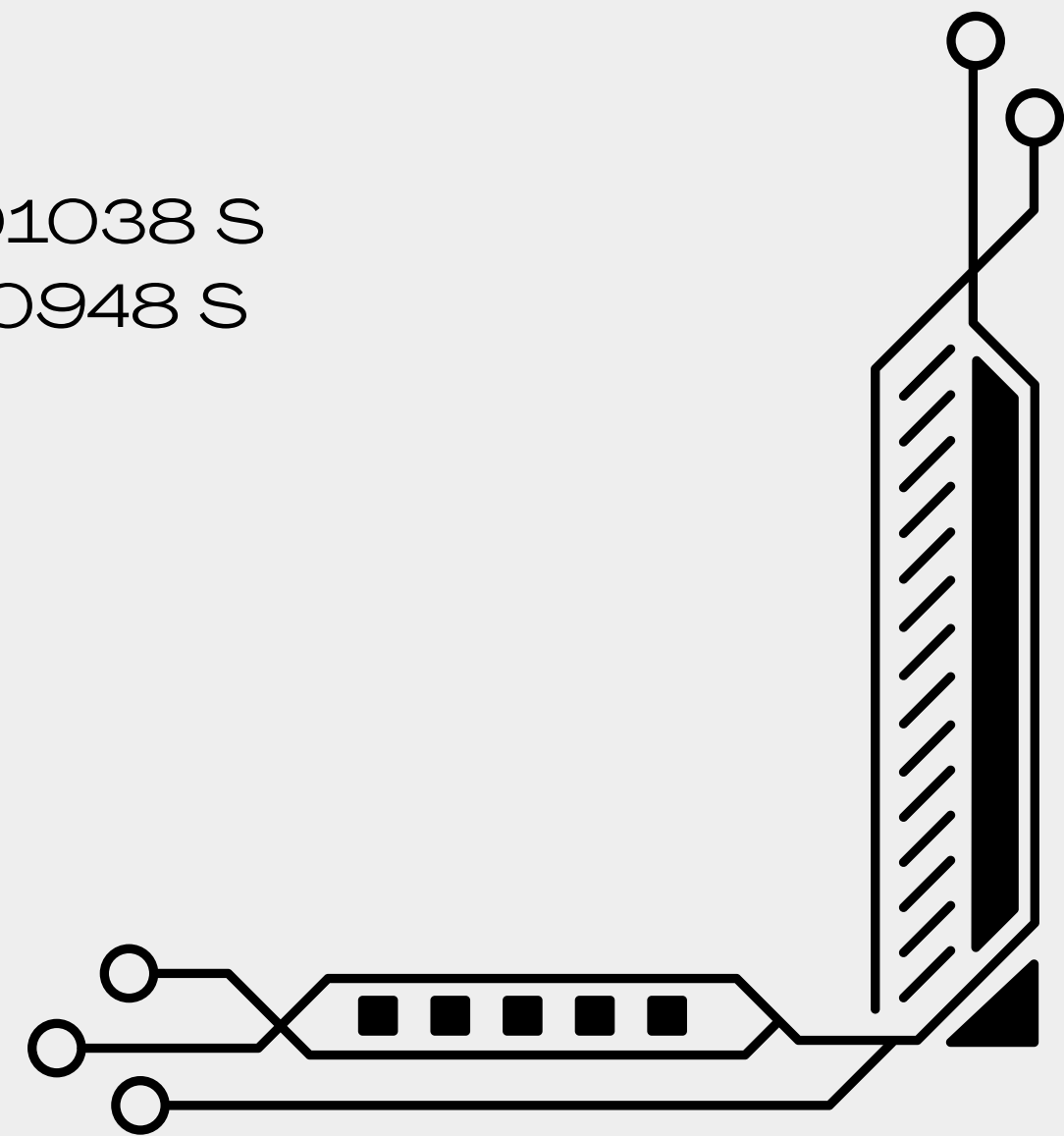
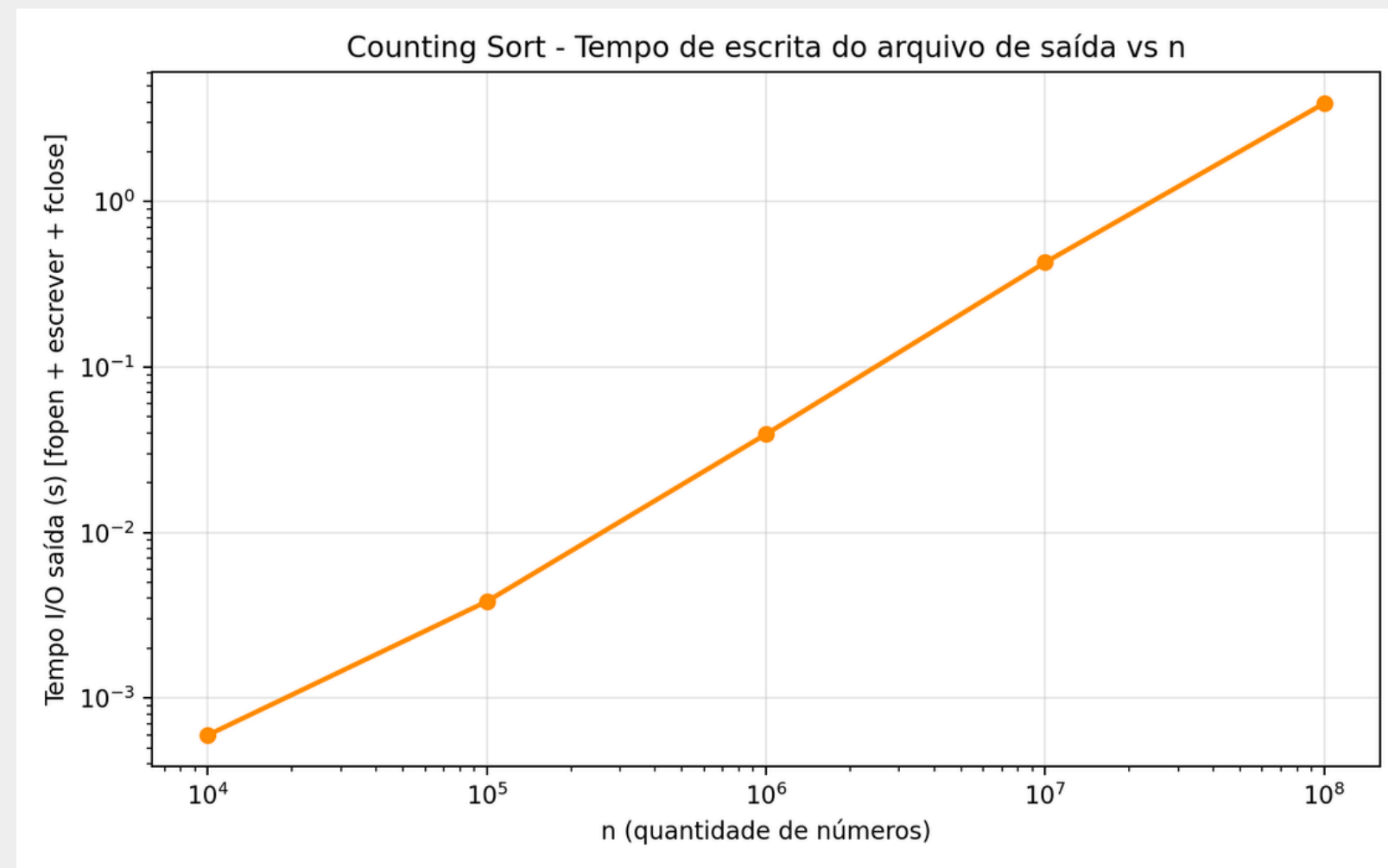
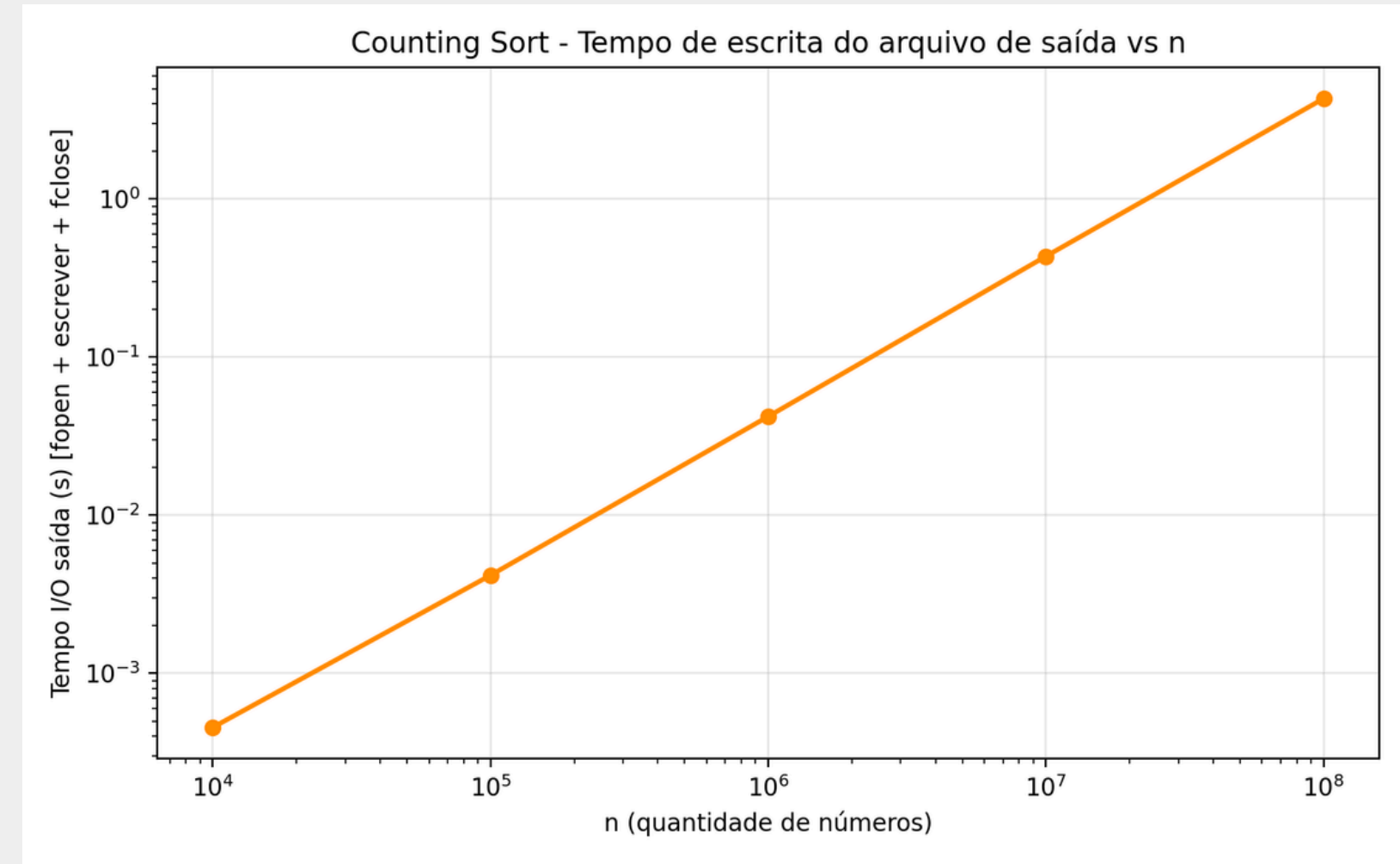


GRÁFICO ESCRITA VS N



K: 10000



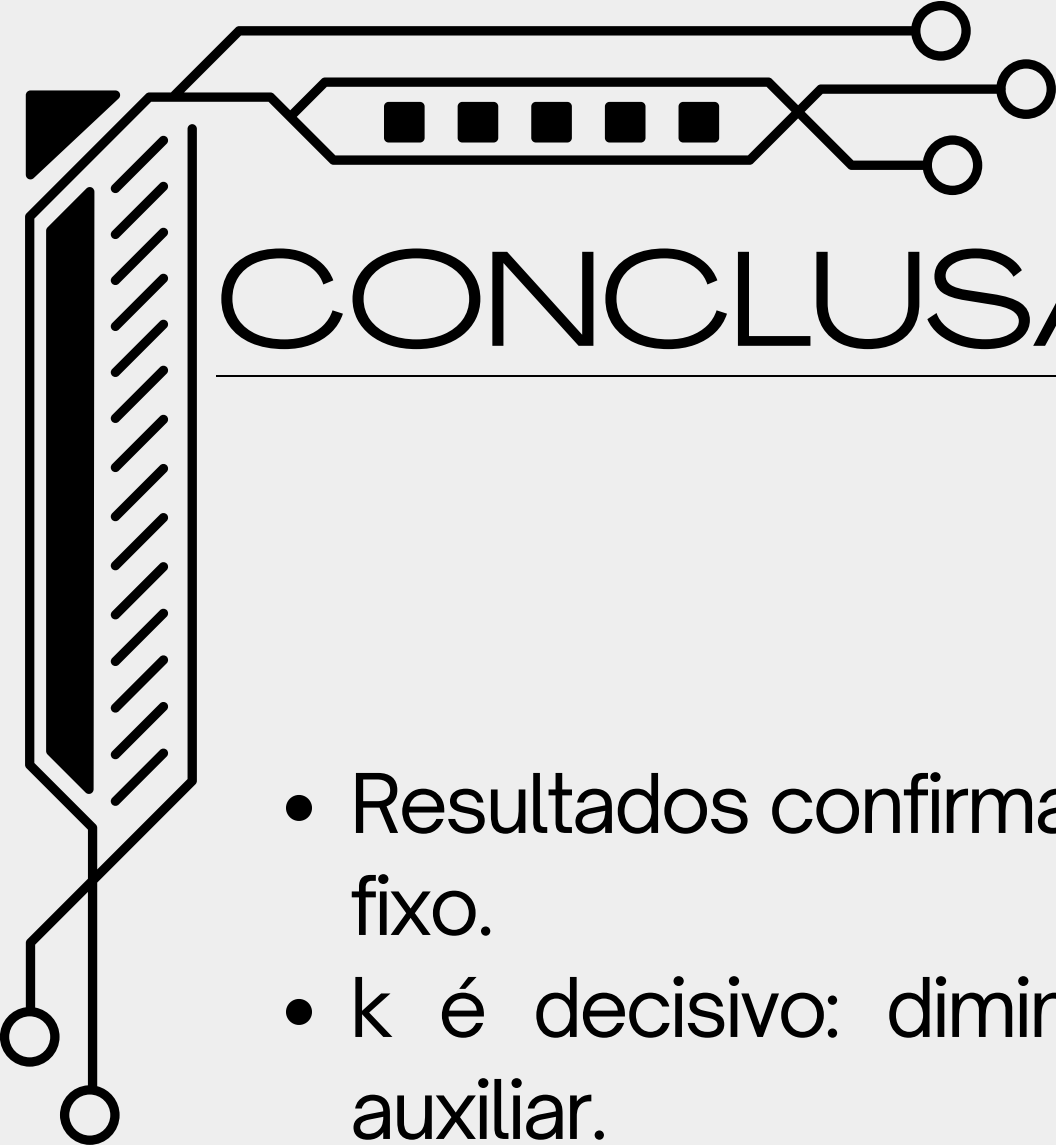
K: 10000000



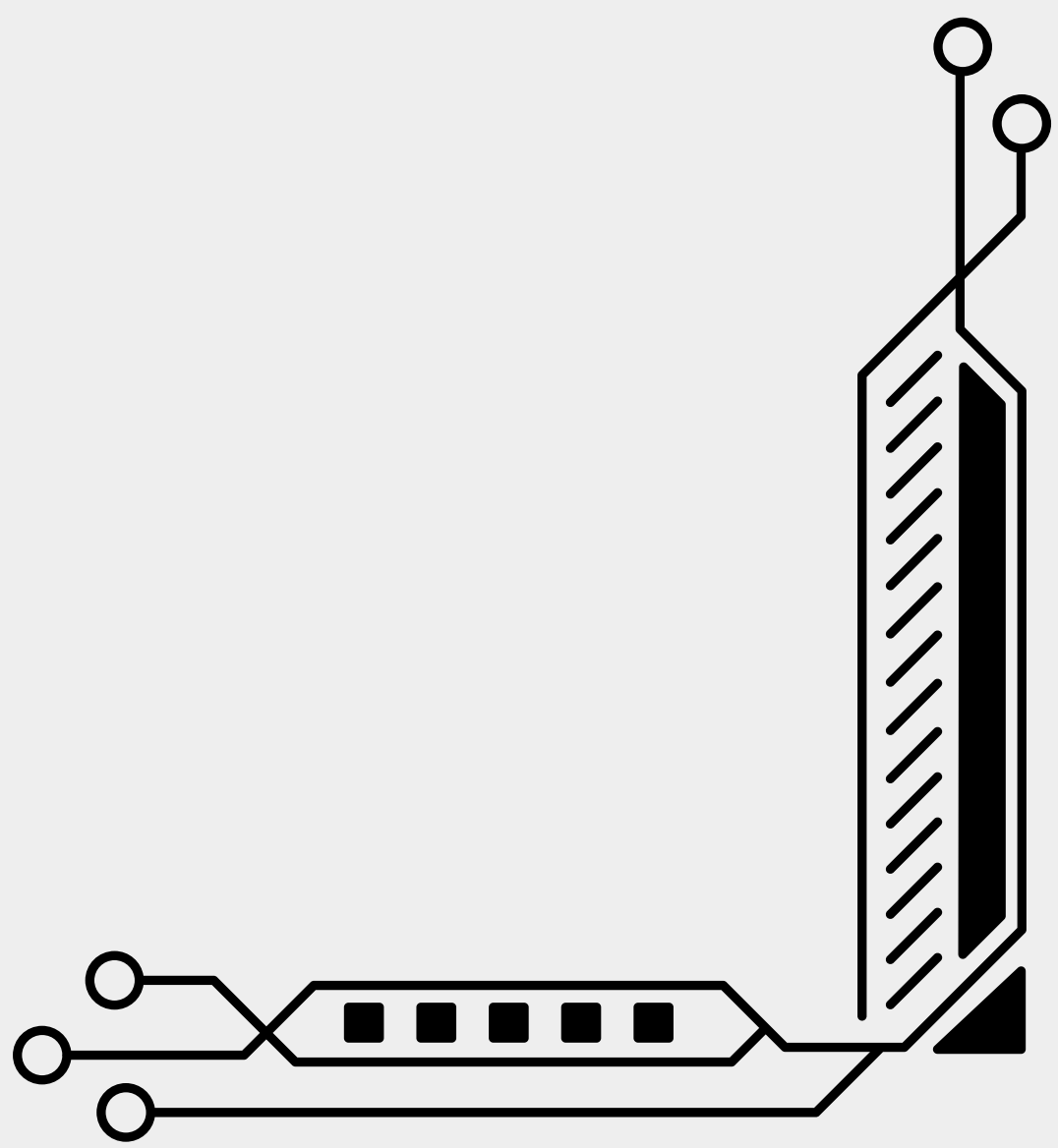
ANÁLISE

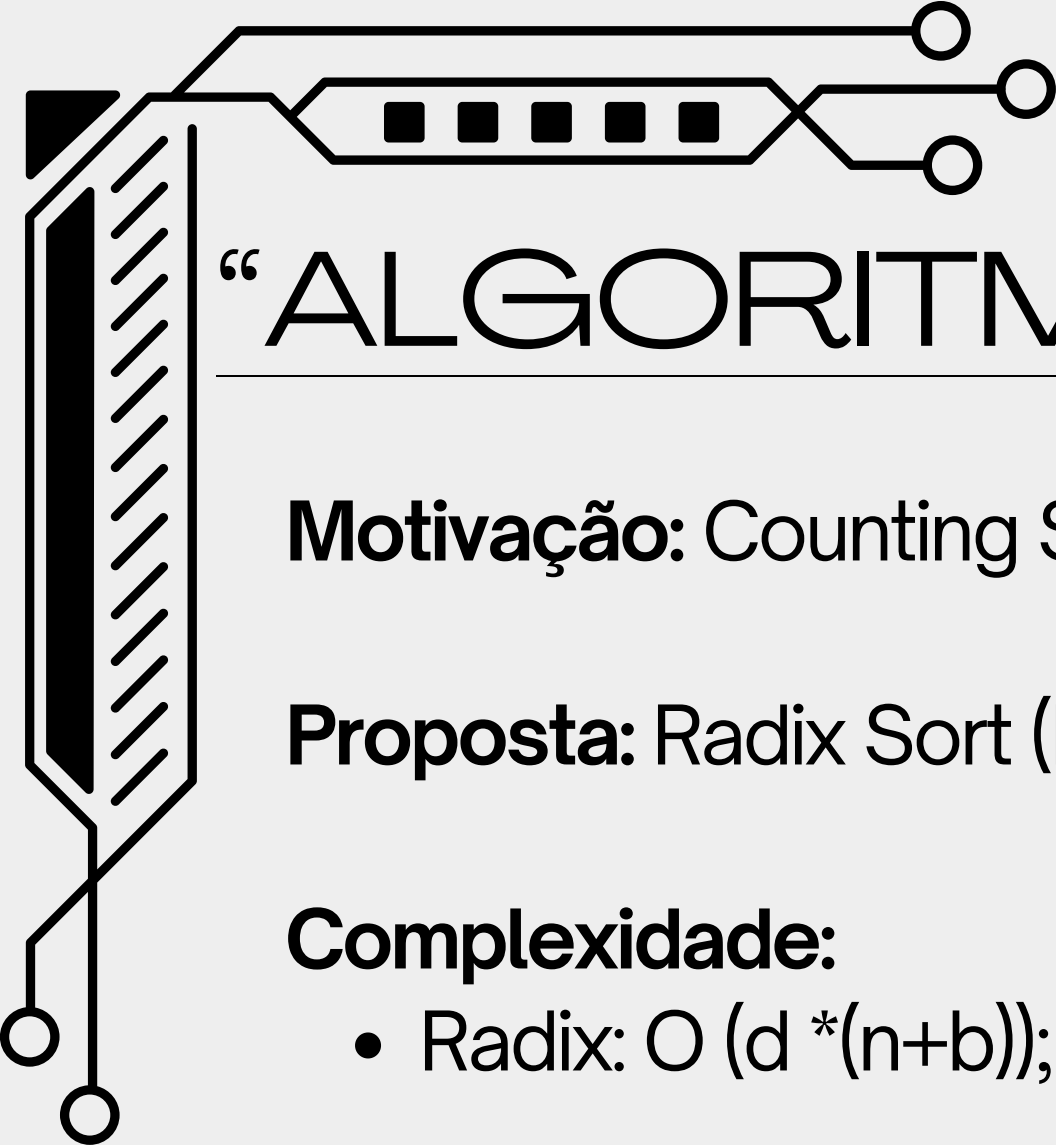
QUAL O COMPORTAMENTO DO COUNTING SORT?

- Teoria: Counting Sort $\rightarrow T(n,k)=O(n+k)$, memória $O(n+k)$.
- Na prática: com k fixo, o tempo cresce quase linear com n .
- Impacto de k : reduzir k (1.000.000 $\rightarrow$ 10.000) reduz o tempo (ex.: 100M: $\sim 0,1188$ s $\rightarrow$ $\sim 0,0671$ s), evidenciando o termo $+k$.
- I/O separado: salvar a saída cresce com n e é quase independente de k (ex.: 100M: $\sim 4,44$ s vs $\sim 3,93$ s).



CONCLUSÃO

- Resultados confirmam $O(n+k)$ e tendência ~linear em n quando k é fixo.
 - k é decisivo: diminuir k melhora desempenho e reduz custo auxiliar.
 - Para n grande, o gargalo pode ser I/O, não a ordenação.
 - Limitação do Counting Sort: k grande; por isso entra o Radix Sort como alternativa.
- 



“ALGORITMO MAIS EFICIENTE”

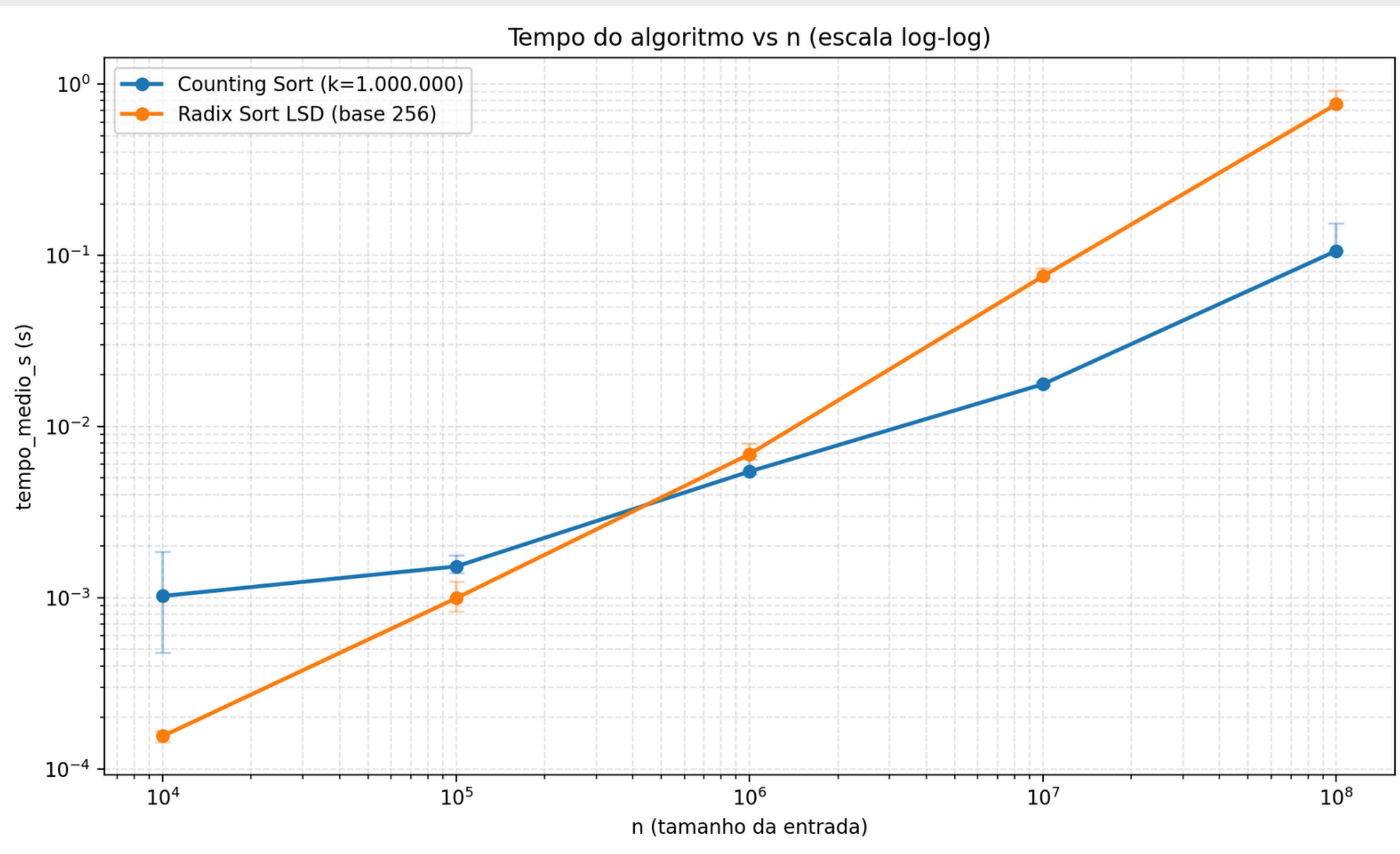
Motivação: Counting Sort é $O(n + k)$. Se K cresce muito, o custo $O(k)$ domina.

Proposta: Radix Sort (LSD) para inteiros, base fixa (ex: 256).

Complexidade:

- Radix: $O(d \cdot (n+b))$;
 - Para inteiros de tamanhos fixo: d constante (ex: 4 passes para 32 bit em massa 256) e b constante $\Rightarrow O(n)$.
 - **Conclusão:** Radix elimina dependência direta de k , sendo eficiente quando k é muito grande
- 

“ALGORITMO MAIS EFICIENTE”



OBRIGADO

